

Smart Lender

Project Description:

Smart Lender is an end-to-end Machine Learning web application designed to predict loan approval decisions. The platform leverages a trained XGBoost Classifier to assess loan applications based on applicant demographic details, credit history, and financial indicators, enabling banks and financial institutions to make faster, data-driven decisions.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, co-applicant income, loan amount, loan term, credit history, and property area. After training and comparing multiple classification algorithms, the best-performing model is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask, backed by a normalized MySQL database, and deployed publicly on Render, Smart Lender provides a seamless interface where applicants can submit their details and instantly receive an approval decision with a confidence score, while credit officers and analysts can review historical predictions through an audit dashboard.

Scenarios

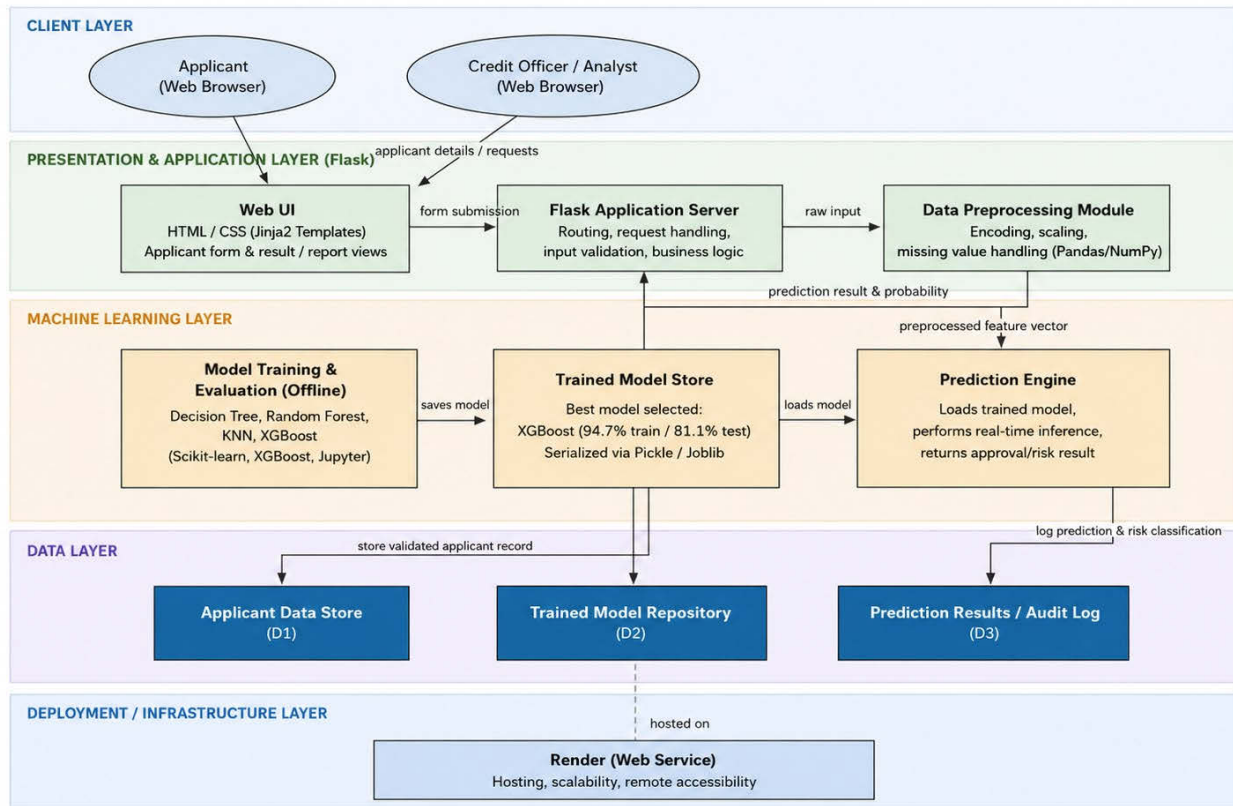
Scenario 1: A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2: An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained model, prompting further scrutiny or document verification before proceeding.

Scenario 3: A financial analyst uses the platform to batch-evaluate multiple applicants during a high-volume period. By relying on the XGBoost model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

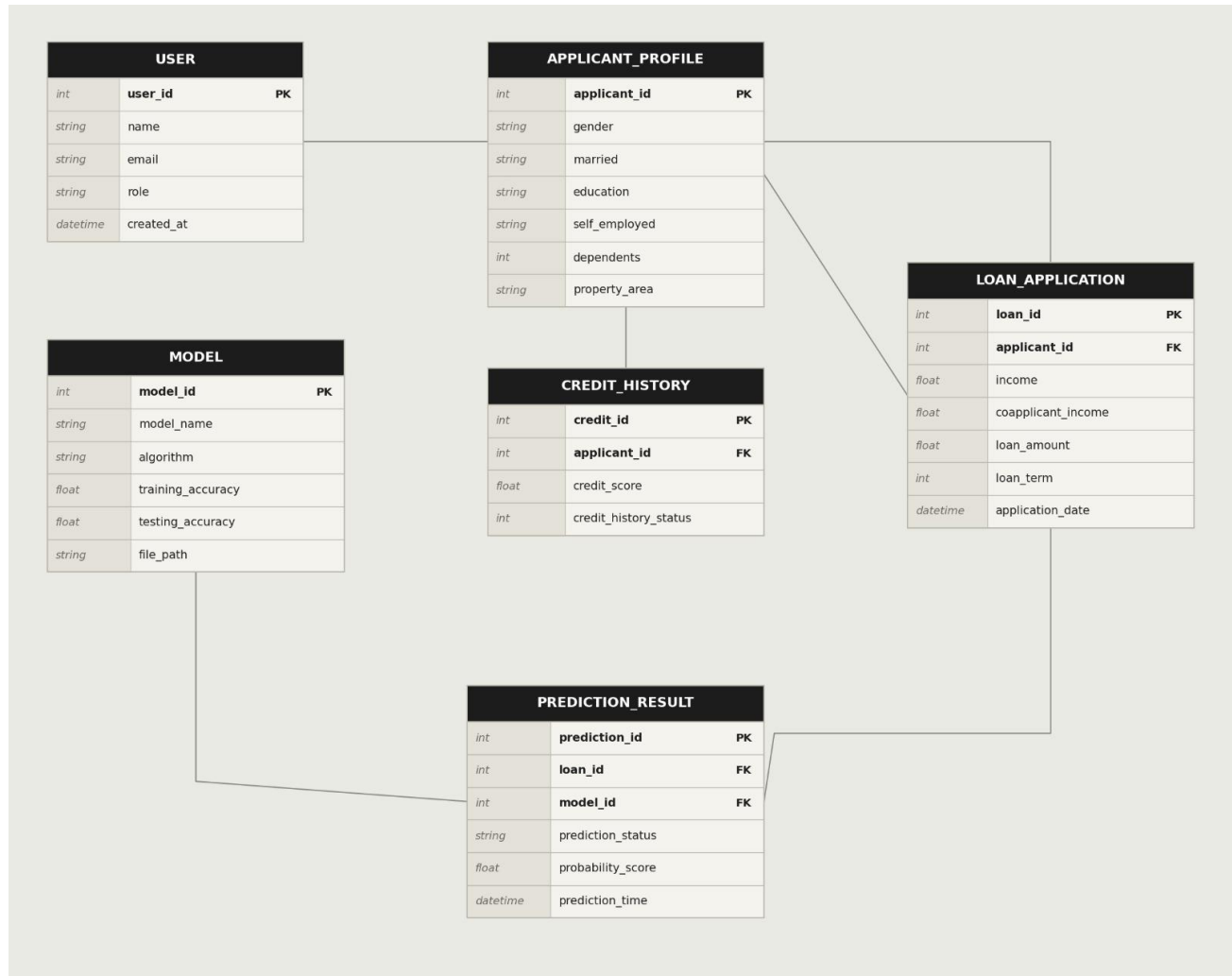
Solution Architecture:

Smart Lender - Solution Architecture



Description: Smart Lender uses a layered architecture to deliver real-time, ML-driven loan predictions. The client layer (applicant and credit officer browsers) interacts with a Flask-based presentation and application layer, which validates and preprocesses input before passing it to the Machine Learning layer. The ML layer trains, stores, and serves the best-performing model (XGBoost) for inference. A MySQL data layer stores applicant records, the model artifact reference, and prediction/audit logs, and the entire application is hosted on Render for public accessibility.

ER Diagram:



Description:

The Smart Lender database uses a normalized relational schema consisting of six tables — **USER**, **APPLICANT_PROFILE**, **CREDIT_HISTORY**, **LOAN_APPLICATION**, **MODEL**, and **PREDICTION_RESULT** — designed to track applicants, their credit standing, loan requests, and prediction outcomes. Each applicant has one associated credit history record and can submit multiple loan applications, with each loan application yielding a single prediction result generated by a specific trained model (e.g., XGBoost), while system users such as credit officers and analysts are tracked separately for auditability. This structure ensures clean separation of applicant data, credit information, loan details, and model-driven predictions, enabling accurate historical tracking and straightforward reporting on which model approved a given loan and with what confidence.

Pre-requisites:

- ✎ Flask Framework Knowledge: [Flask Documentation \(flask.palletsprojects.com\)](https://flask.palletsprojects.com/)
- ✎ MySQL Database Familiarity: [MySQL Documentation \(dev.mysql.com/doc\)](https://dev.mysql.com/doc/)
- ✎ Machine Learning Fundamentals: [Scikit-learn Documentation \(scikit-learn.org\)](https://scikit-learn.org/)
- ✎ XGBoost Familiarity: [XGBoost Documentation \(xgboost.readthedocs.io\)](https://xgboost.readthedocs.io/)
- ✎ HTML, CSS, and JavaScript Skills: [W3Schools Tutorials \(w3schools.com\)](https://www.w3schools.com/)

- ✎ Python Programming Proficiency: [Python Documentation \(docs.python.org\)](https://docs.python.org)
- ✎ Version Control with Git: [Git Documentation \(git-scm.com/docs/git\)](https://git-scm.com/docs/git)
- ✎ Load Testing with Locust: [Locust Documentation \(locust.io\)](https://locust.io)

Project Workflow:

Milestone 1: Data Collection, Preprocessing & Model Selection

- Activity 1.1: Collect and explore the loan applicant dataset (614 records) covering demographics, income, credit history, and property area.
- Activity 1.2: Preprocess the data — encode categorical variables, impute missing values using mode, treat outliers using the IQR method, and balance the target classes using SMOTE.
- Activity 1.3: Train and compare Decision Tree, KNN, Random Forest, Gradient Boosting, and XGBoost classifiers using a 67%-33% train-test split with 5-fold cross-validation.
- Activity 1.4: Select the best-performing model (XGBoost) and serialize it along with the StandardScaler for reuse in real-time prediction.

Milestone 2: Database Design

- Activity 2.1: Design a normalized (3NF) relational schema covering Applicant Profile, Credit History, Loan Application, Model, and Prediction Result entities.
- Activity 2.2: Implement the schema in MySQL and configure the Flask application to connect via environment variables (.env).

Milestone 3: Flask Backend Development

- Activity 3.1: Implement core routes — / (dashboard and history), /submit (application form), and /predict (real-time inference and result display).
- Activity 3.2: On form submission, encode and scale inputs using the saved scaler, run inference with the XGBoost model, store the transaction across the MySQL tables, and return the prediction with a confidence score.

Milestone 4: Frontend Development

- Activity 4.1: Design the Smart Lender dashboard, showing an overview panel and recent audit logs of past predictions.
- Activity 4.2: Build the New Application Check form, capturing gender, marital status, dependents, education, employment, credit history, property area, income, and loan details.
- Activity 4.3: Build the prediction result page, displaying the approval/rejection decision, confidence percentage, and a summary of the submitted applicant details.

Milestone 5: Deployment & Performance Testing

- ✎ Activity 5.1: Set up a virtual environment, install dependencies from requirements.txt, configure the MySQL database and .env file, and run the app locally with python Flask/app1.py.
- ✎ Activity 5.2: Deploy the application publicly on Render, making it accessible at a live URL without local hosting.
- ✎ Activity 5.3: Conduct load testing using Locust to measure response times, throughput (requests per second), and behavior under concurrent user load.

Milestone 6: Conclusion

Milestone 6 marks the culmination of the Smart Lender project, bringing together data preprocessing, model training and selection, database design, backend development, frontend design, and deployment into a single, functioning end-to-end system. Through this milestone, the trained XGBoost model, integrated Flask application, and normalized MySQL database were validated as a cohesive unit — confirming that applicants can submit their details and receive real-time, accurate loan approval predictions, while credit officers and analysts can review historical decisions through the audit dashboard. Performance testing further confirmed the application's stability under concurrent user load, ensuring the platform is ready for demonstration and evaluation. This milestone reflects the project's overall success in transforming a traditionally manual, time-consuming credit evaluation process into a fast, consistent, and data-driven decision-support system.

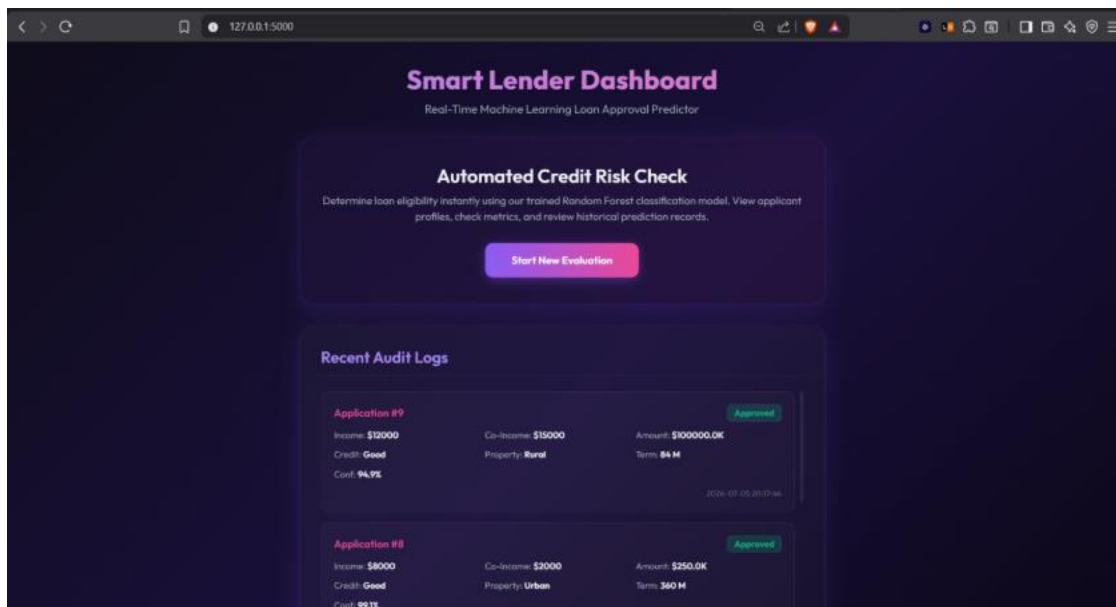
Milestone 1: Data Collection, Preprocessing & Model Selection

This milestone focuses on preparing a reliable, model-ready dataset and identifying the best-performing classification algorithm for predicting loan approval.

Activity 1.1: Dataset Collection & Exploration

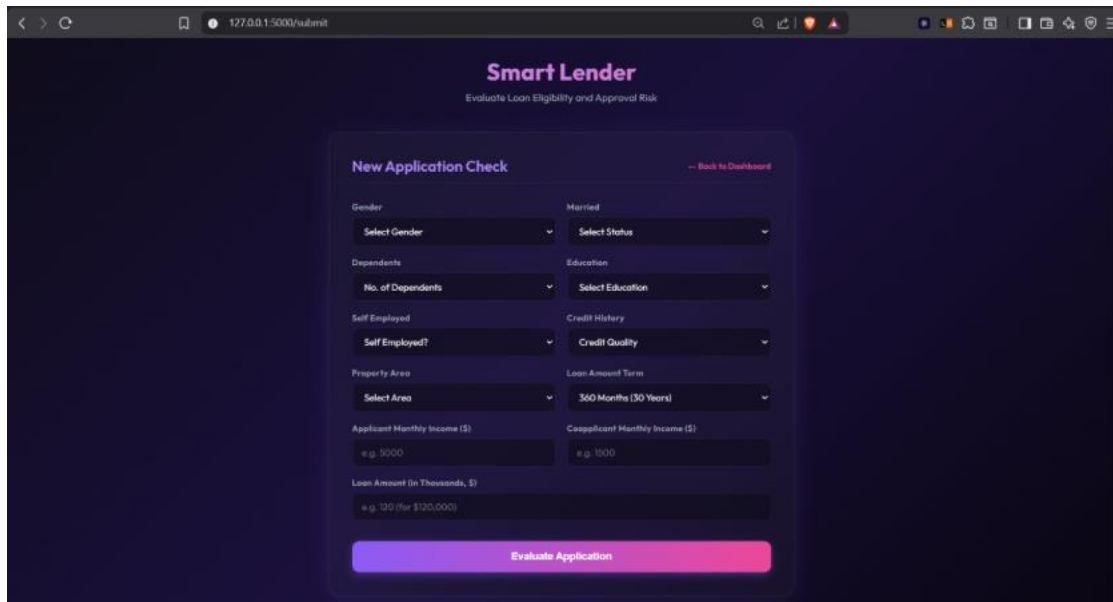
The model uses the loan_prediction.csv dataset (614 records) containing demographic features (Gender, Married, Dependents, Education, Self_Employed), financial features (ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term), and credit/location features (Credit_History, Property_Area), with Loan_Status as the target class. Exploratory Data Analysis — correlation heatmaps, boxplots, and income/credit histograms — is performed to understand feature relationships and detect anomalies.

[Home / Dashboard Page:](#)



Description: The Smart Lender Dashboard serves as the landing page, featuring a bold gradient title, a tagline ('Real-Time Machine Learning Loan Approval Predictor'), and an 'Automated Credit Risk Check' panel with a 'Start New Evaluation' call-to-action button. Below it, a scrollable Recent Audit Logs section lists past applications with their income details, credit status, confidence score, and approval outcome, each timestamped for auditability.

New Application Check — Input Form:



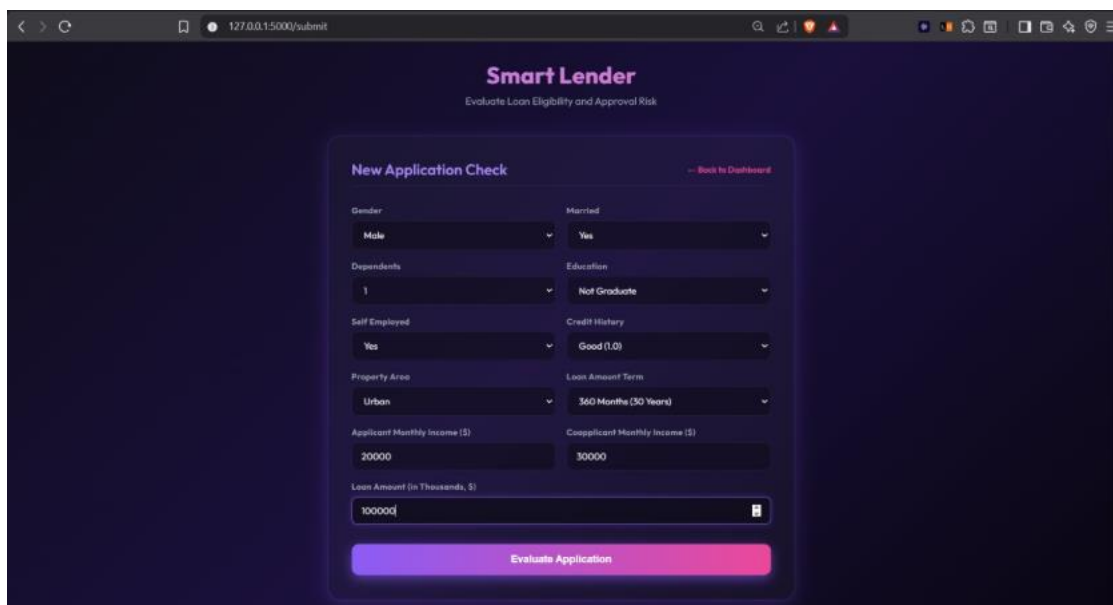
The screenshot shows the 'New Application Check' form in the Smart Lender interface. The form is titled 'New Application Check' with a link to 'Back to Dashboard'. It contains several input fields arranged in two columns:

- Gender:** Select Gender (dropdown)
- Married:** Select Status (dropdown)
- Dependents:** No. of Dependents (dropdown)
- Education:** Select Education (dropdown)
- Self Employed:** Self Employed? (dropdown)
- Credit History:** Credit Quality (dropdown)
- Property Area:** Select Area (dropdown)
- Loan Amount Term:** 360 Months (30 Years) (dropdown)
- Applicant Monthly Income (\$):** e.g. 5000 (text input)
- Coapplicant Monthly Income (\$):** e.g. 1000 (text input)
- Loan Amount (in Thousands, \$):** e.g. 120 (for \$120,000) (text input)

At the bottom of the form is a large blue button labeled 'Evaluate Application'.

Description: The application submission form allows the user to enter applicant details across two columns — Gender, Married, Dependents, Education, Self Employed, and Credit History on one side, with Property Area and Loan Amount Term on the other — followed by Applicant/Co-applicant Monthly Income and Loan Amount fields. Placeholder text (e.g., 'e.g. 5000') guides correct input format before the user clicks 'Evaluate Application'. **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select “API Keys” from the dropdown menu, or go directly to the [API Keys page](#).

New Application Check — Filled Example:



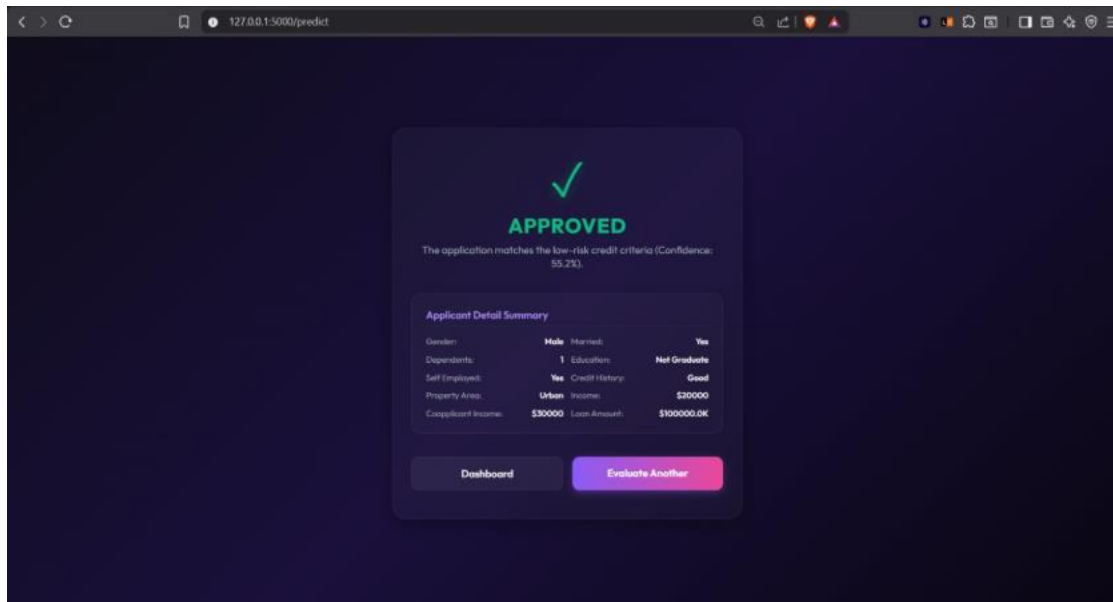
The screenshot shows the 'New Application Check' form filled with example data. The form is titled 'New Application Check' with a link to 'Back to Dashboard'. The filled data is as follows:

- Gender:** Male
- Married:** Yes
- Dependents:** 1
- Education:** Not Graduate
- Self Employed:** Yes
- Credit History:** Good (1.0)
- Property Area:** Urban
- Loan Amount Term:** 360 Months (30 Years)
- Applicant Monthly Income (\$):** 20000
- Coapplicant Monthly Income (\$):** 30000
- Loan Amount (in Thousands, \$):** 100000

At the bottom of the form is a large blue button labeled 'Evaluate Application'.

Description: An example of a completed application: a married, self-employed male applicant with 1 dependent, not a graduate, good credit history, and property in an urban area, requesting a loan against a combined applicant and co-applicant income of \$50,000.

Prediction Result Page:



Description: After evaluation, the result page displays a green checkmark with an 'APPROVED' status and a short explanation referencing the confidence score (e.g., 55.2%). Below it, an Applicant Detail Summary card recaps all submitted fields for the user's reference, followed by 'Dashboard' and 'Evaluate Another' action buttons.

Activity 1.2: Data Preprocessing

- ✎ Categorical Encoding: mapping features to numerical scales (e.g., Graduate: 1, Not Graduate: 0; Urban: 2, Semiurban: 1, Rural: 0).
- ✎ Missing Value Imputation: filling missing values using the mode of each feature.
- ✎ Outlier Treatment: handling outliers in ApplicantIncome, CoapplicantIncome, and LoanAmount using the Interquartile Range (IQR) method.
- ✎ Feature Scaling: standardizing features using StandardScaler for scale-invariant model training.
- ✎ Class Imbalance Mitigation: the original dataset had 422 Approved : 192 Rejected records; SMOTE was applied to balance it to 422 : 422, preventing bias toward approvals.

Activity 1.3: Model Training & Comparison

Multiple algorithms were trained and evaluated using a 67%-33% train-test split with 5-fold cross-validation for generalizability:

- ✎ Decision Tree — 78.70% test accuracy, 81.30% CV mean accuracy (max depth constrained to 5 to avoid overfitting).
- ✎ KNN (K-Nearest Neighbors) — 78.11% test accuracy, 78.40% CV mean accuracy (5 neighbors, Euclidean distance).
- ✎ Random Forest — 84.02% test accuracy, 84.80% CV mean accuracy (100-tree ensemble on

balanced data).

- ✧ Gradient Boosting — 84.02% test accuracy, 85.10% CV mean accuracy (iterative boosting of trees).
- ✧ XGBoost (XGBClassifier) — 84.94% test accuracy, 85.34% CV mean accuracy — selected as the optimal model for its high accuracy and low variance.

Activity 1.4: Model Selection & Serialization

The final XGBoost model is saved as `Flask/rdf.pkl` (named for backward compatibility with the application shell), and the corresponding `StandardScaler` is saved as `Flask/scale1.pkl`. Both artifacts are loaded by the Flask application at runtime to perform real-time inference without retraining.

Milestone 2: Database Design

Smart Lender implements a relational MySQL database matching an Entity-Relationship Diagram with five core entities: User, Applicant Profile, Credit History, Loan Application, Model, and Prediction Result. Applicant Profile relates one-to-one with Credit History, one-to-many with Loan Application, each Loan Application yields one Prediction Result, and each Model can generate many Prediction Results. The schema (`Flask/schema.sql`) is created in MySQL, and the Flask application connects to it using credentials defined in a `.env` file.

Milestone 3: Flask Backend Development

The Flask application (`Flask/app1.py`) implements the following routes and logic:

- ✧ Dashboard & History (`/`): reads records from the database using structured JOIN queries across the applicant, loan, credit, and prediction tables, displaying probability scores and statuses.
- ✧ Submit Form (`/submit`): collects applicant demographics, income details, loan request, and credit record through a web form.
- ✧ Real-Time Inference (`/predict`): reads form parameters, encodes and scales inputs using the saved `StandardScaler`, performs inference using the XGBoost model to yield a prediction class (Approved/Rejected) and confidence score, inserts the transaction across the MySQL tables, and displays the result to the user.

Milestone 4: Frontend Development

The frontend uses a dark, glassmorphism-inspired design with a vibrant purple-to-pink accent palette — green highlights for approved outcomes and clear typography for readability.

Activity 4.1: Dashboard Page

The dashboard presents an 'Automated Credit Risk Check' panel with a call-to-action to start a new evaluation, followed by a scrollable list of recent audit logs showing each application's income, co-applicant income, loan amount, credit status, property area, term, confidence score, and approval outcome.

Activity 4.2: Application Submission Form

The 'New Application Check' form captures Gender, Married status, Dependents, Education, Self Employed status, Credit History, Property Area, Loan Amount Term, Applicant/Co-applicant Monthly Income, and Loan Amount, with an 'Evaluate Application' button to submit the request.

Activity 4.3: Prediction Result Page

Upon submission, the result page displays a clear Approved/Rejected badge with a confidence percentage, a short explanation of the decision, a summary of the submitted applicant details, and options to return to the Dashboard or evaluate another application.

Milestone 5: Deployment & Performance Testing

Activity 5.1: Local Deployment

- ✎ Create a virtual environment: `python -m venv .venv`
- ✎ Activate it (Windows PowerShell): `.venv\Scripts\Activate.ps1`
- ✎ Install dependencies: `pip install -r requirements.txt`
- ✎ Create the MySQL database: `CREATE DATABASE IF NOT EXISTS smart_lender;`
- ✎ Import the schema: `mysql -u root -p smart_lender < Flask/schema.sql`
- ✎ Configure the `.env` file with `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, and `DB_NAME`.
- ✎ Run the application: `python Flask/app1.py`, then open <http://127.0.0.1:5000/>

Activity 5.2: Public Deployment

The application is deployed publicly on Render, making it accessible at <https://smart-lender-dple.onrender.com> without requiring local hosting.

Activity 5.3: Performance & Load Testing

Load testing was carried out using Locust to evaluate the application's behavior under concurrent user load, simulating up to 50 concurrent users and measuring requests per second, failure rate, and response time percentiles.



Conclusion:

Smart Lender is a machine learning-powered web application that streamlines loan approval decisions using a trained XGBoost classifier. It processes applicant demographic and financial data through a preprocessing pipeline (encoding, IQR outlier handling, and SMOTE balancing), stores records in a normalized MySQL database, and serves real-time predictions with confidence scores through a Flask web interface. Performance testing with Locust validated the system's behavior under concurrent load and highlighted opportunities for scaling optimization. Looking ahead, Smart Lender offers potential for further enhancement — such as adding authentication and role-based access, retraining on larger real-world datasets, and improving infrastructure for production-grade scalability — positioning it to evolve from a prototype into a robust decision-support tool for banks and financial institutions.